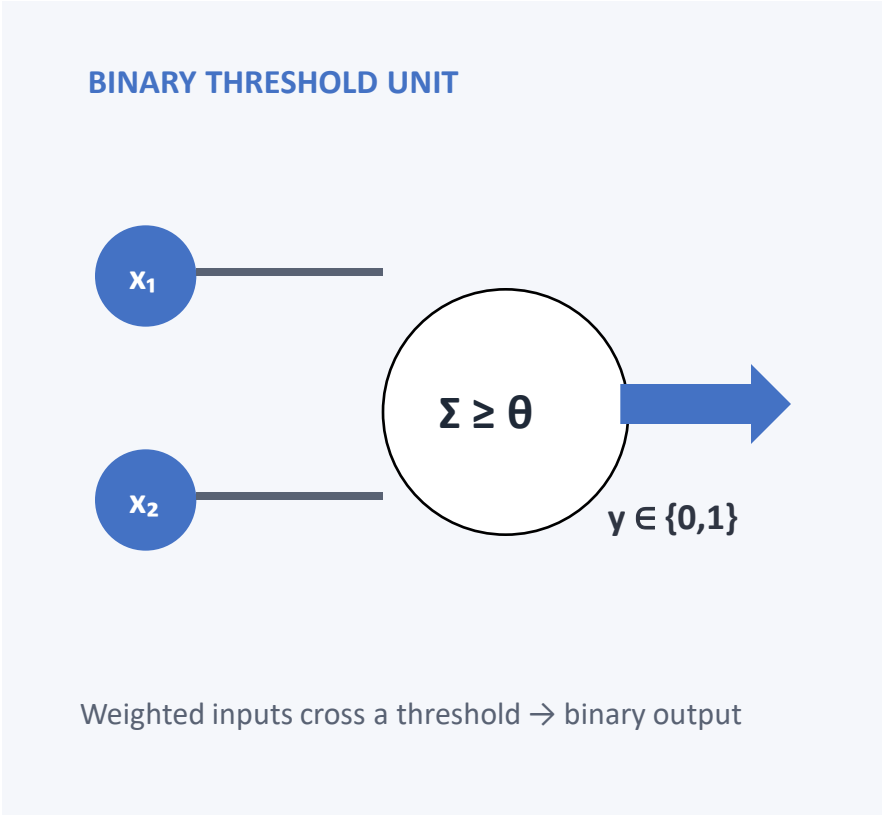
Lecture 2

The Rise, Fall, and Return of Neural Networks

History, The First Artificial Neuron

1943

- **Warren McCulloch and Walter Pitts**
  - Proposed the first influential mathematical model of an artificial neuron.
  - The unit summed binary inputs and activated when a threshold was reached.
  - This showed that neural networks could be studied mathematically.

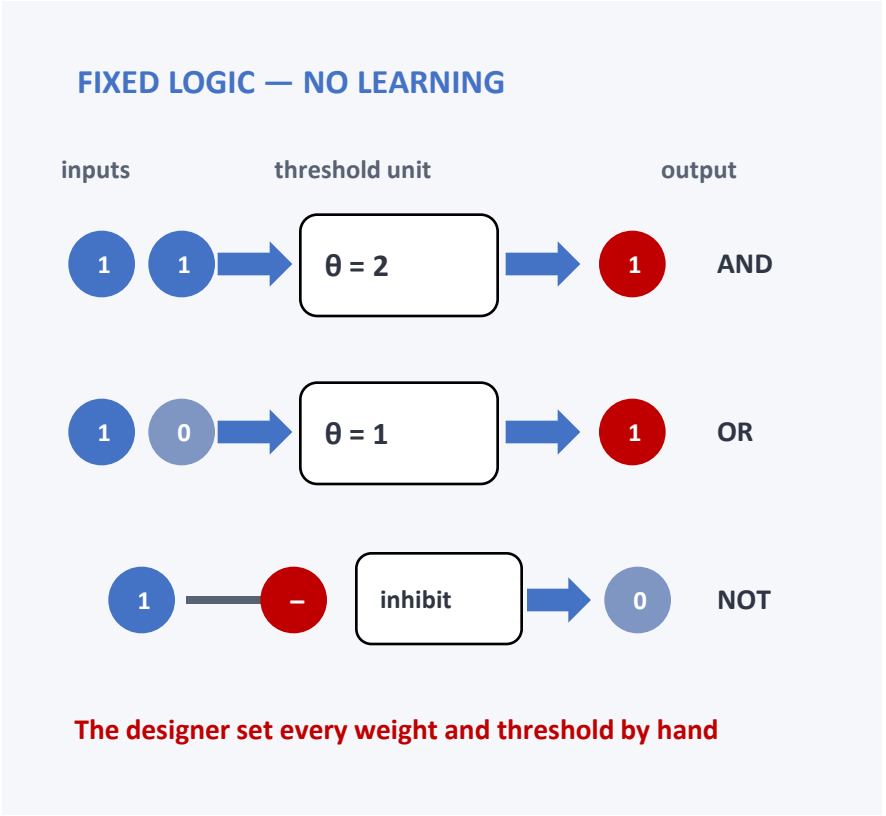


History, From Neurons to Logic

1943

What could the model do?

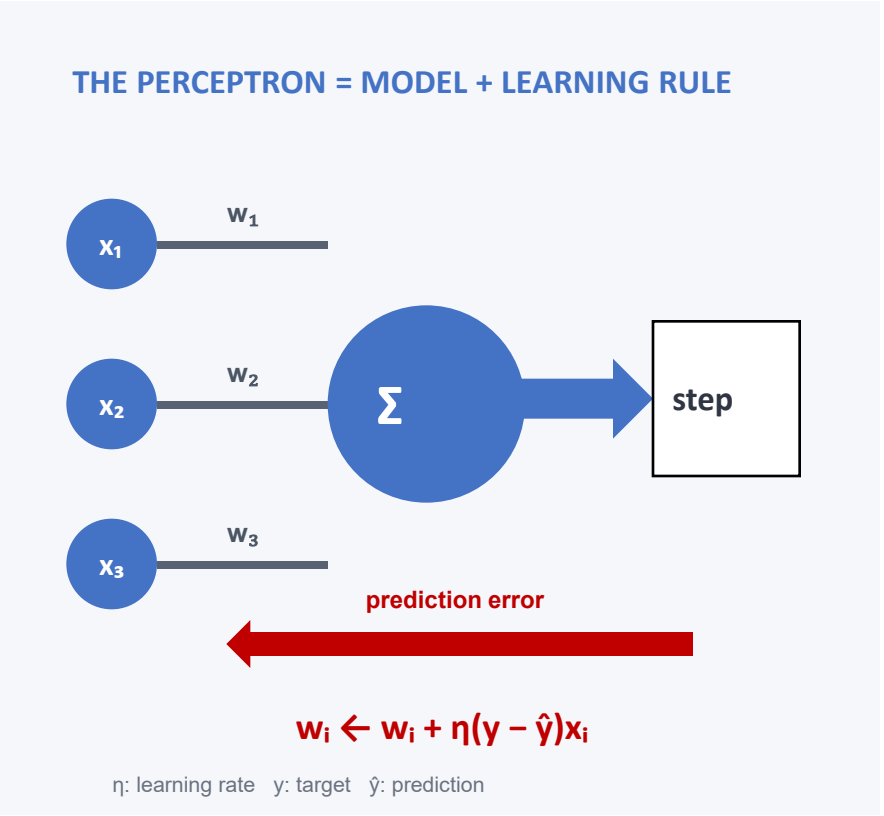
- Different thresholds and inhibitory inputs produced AND, OR, and NOT operations.
- Networks of neurons could therefore represent Boolean computations.
- **But the weights and thresholds were fixed—the network did not learn.**



History, Rosenblatt Invents the Perceptron

1957–1958

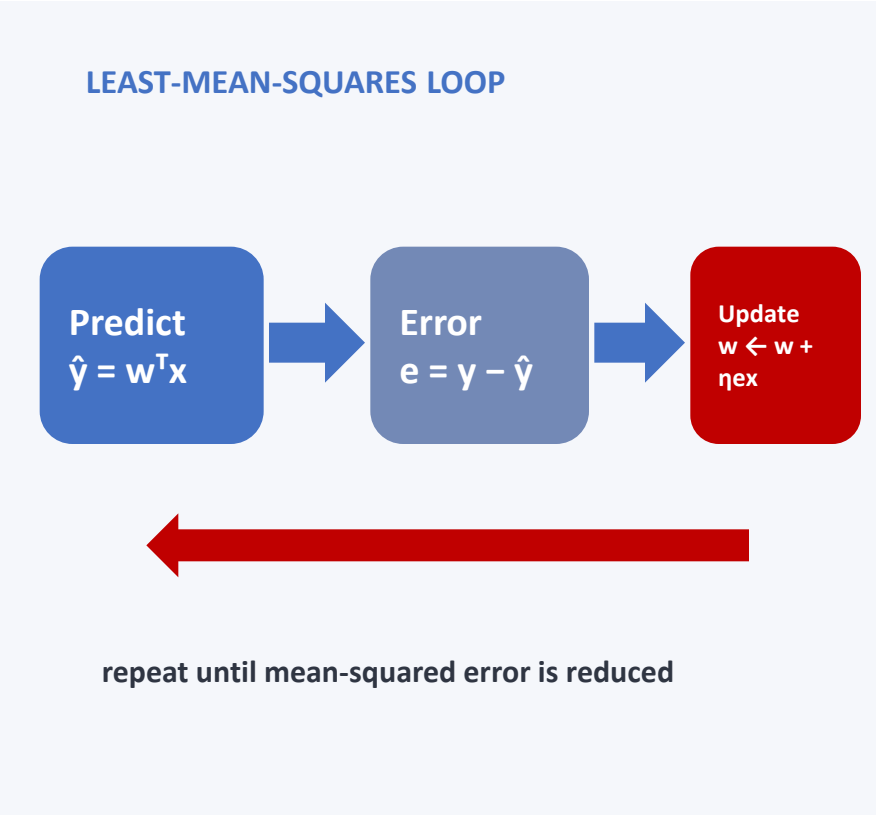
- **Frank Rosenblatt invented the perceptron**
  - A weighted sum of the inputs passed through a threshold to make a prediction.
  - Unlike the 1943 model, its input weights were adjusted from labeled examples.
  - The Mark I Perceptron demonstrated this learning procedure in hardware.



History, Optimizing the Weights

1960

- **Bernard Widrow and Marcian Hoff**
  - Introduced ADALINE and the least-mean-squares learning rule.
  - Training became an optimization loop: predict, measure error, update the weights, and repeat.
  - This established a practical gradient-based method for learning weights.



History, The Perceptron Dream

1958–1960s

The ability to learn created enormous optimism.

- Press reports predicted machines that could see, speak, write, and become conscious.
- Many expected perceptron-like systems to solve broad artificial-intelligence problems.
- **The expectations grew much faster than the technology.**

THE PERCEPTRON DREAM

"an electronic computer that the Navy expects will be able to

walk, talk, see, write, reproduce itself

and be conscious of its existence."

The New York Times · July 8, 1958

Public expectations ran far ahead of capability

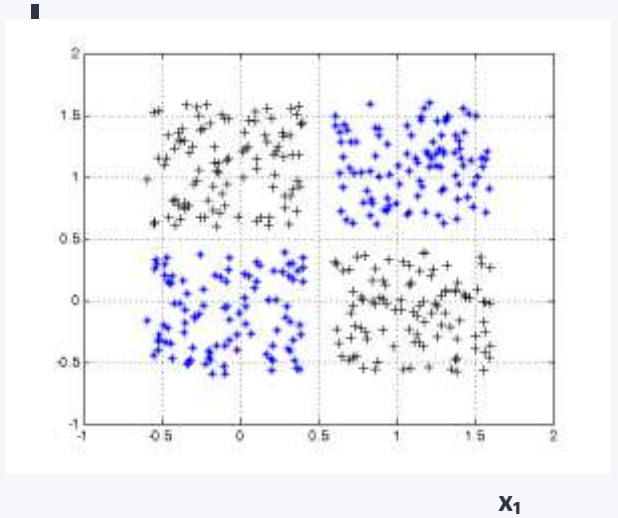
History, The Limitation of Single-Layer Perceptrons

1969

Marvin Minsky and Seymour Papert

- Showed important limitations of single-layer perceptrons.
- A single linear boundary cannot represent the XOR function.
- Multilayer networks could solve XOR, but an effective training method was not yet widely available.
- **The gap between the dream and reality became clear.**

XOR IS NOT LINEARLY SEPARABLE



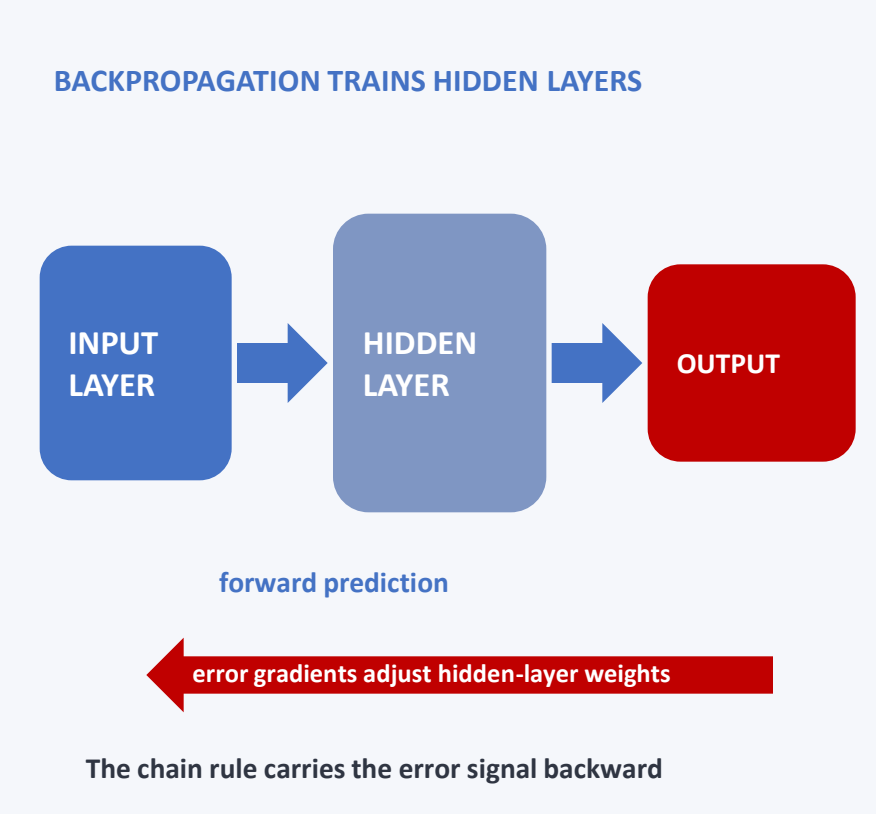
No single straight boundary works

History, Backpropagation Brings Neural Networks Back

1974–1986

The missing piece was training hidden layers.

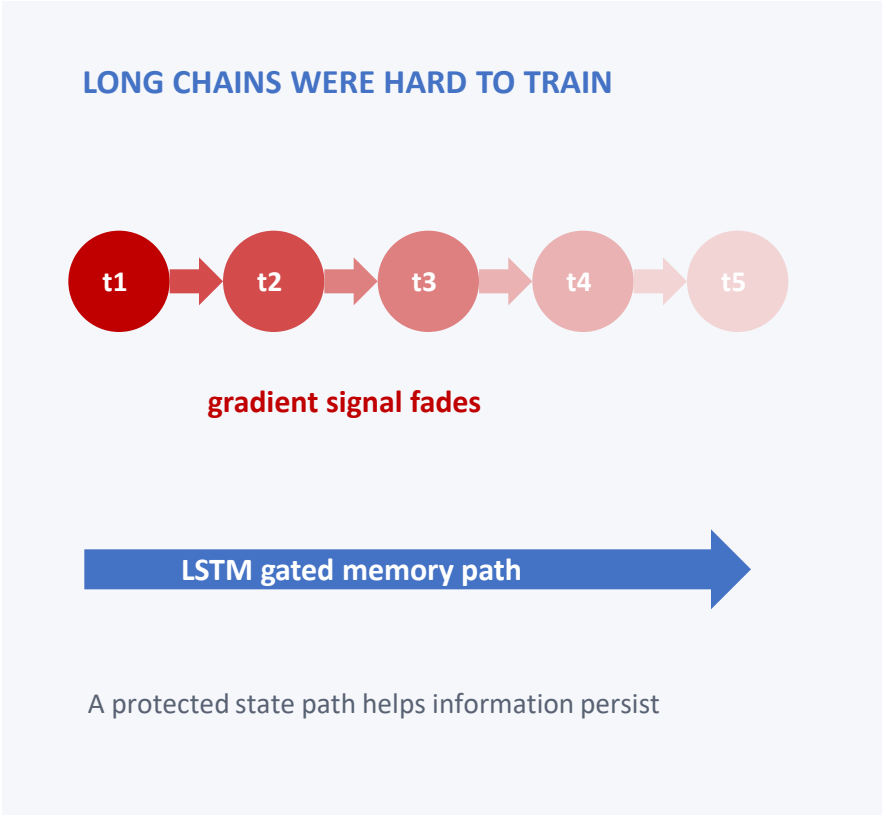
- Paul Werbos described backward gradient computation for multilayer models in 1974.
- Rumelhart, Hinton, and Williams popularized backpropagation in 1986.
- **Errors could now update hidden-layer weights, launching a connectionist revival.**



History, Popularity Fades Again

1990s–early 2000s

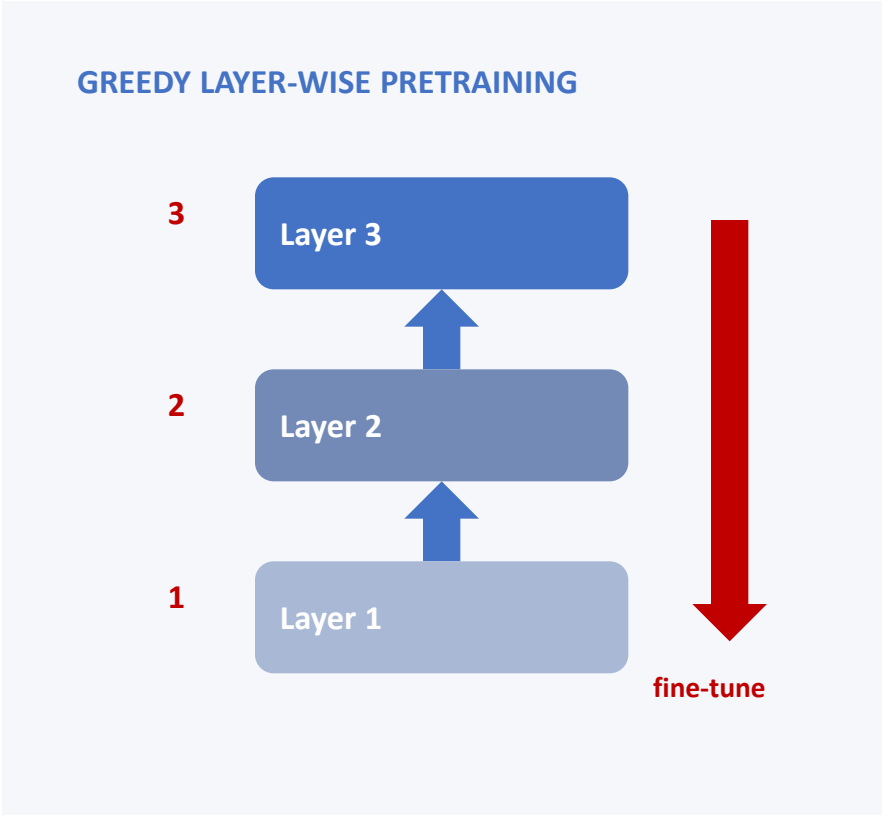
- **Neural networks again lost relative popularity.**
  - Deep and recurrent networks remained difficult to optimize, partly because gradients vanished.
  - Datasets and computing power were limited, while methods such as SVMs performed strongly.
  - Research continued through CNNs and LSTM, but neural networks were no longer the dominant approach.



History, Deep Learning Resurgence

2006–2011

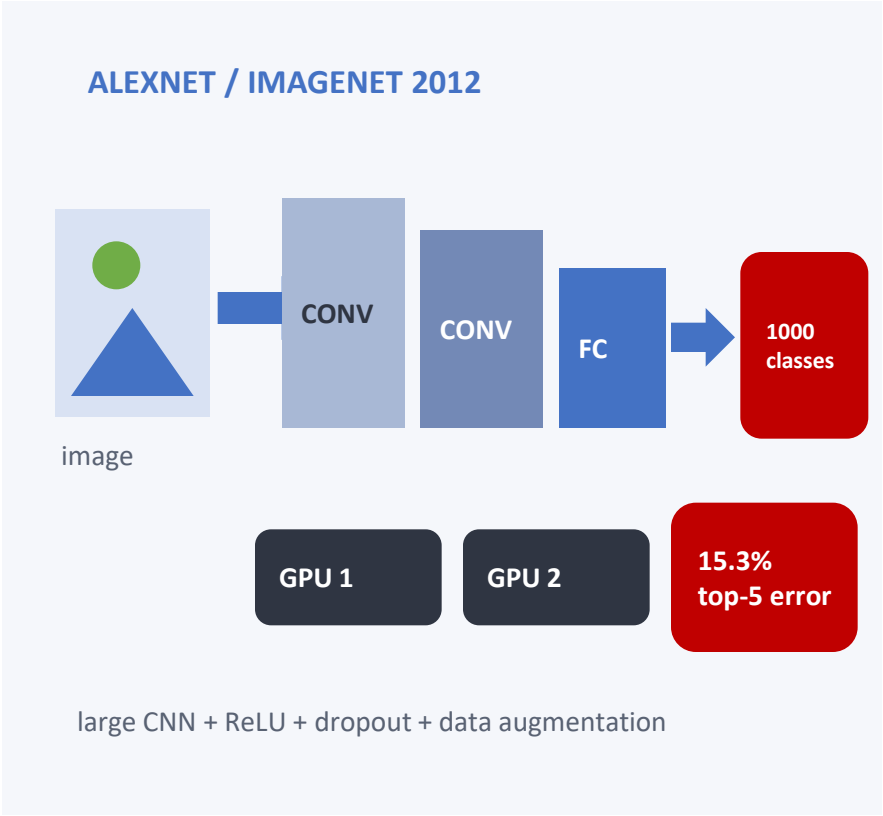
- **Several developments brought deep networks back.**
  - Layer-wise pretraining made deep models easier to initialize and optimize.
  - Larger datasets, faster GPUs, better activations, and improved training methods removed earlier barriers.
  - **Deep learning was ready for a large-scale demonstration.**



History, AlexNet Changes Computer Vision

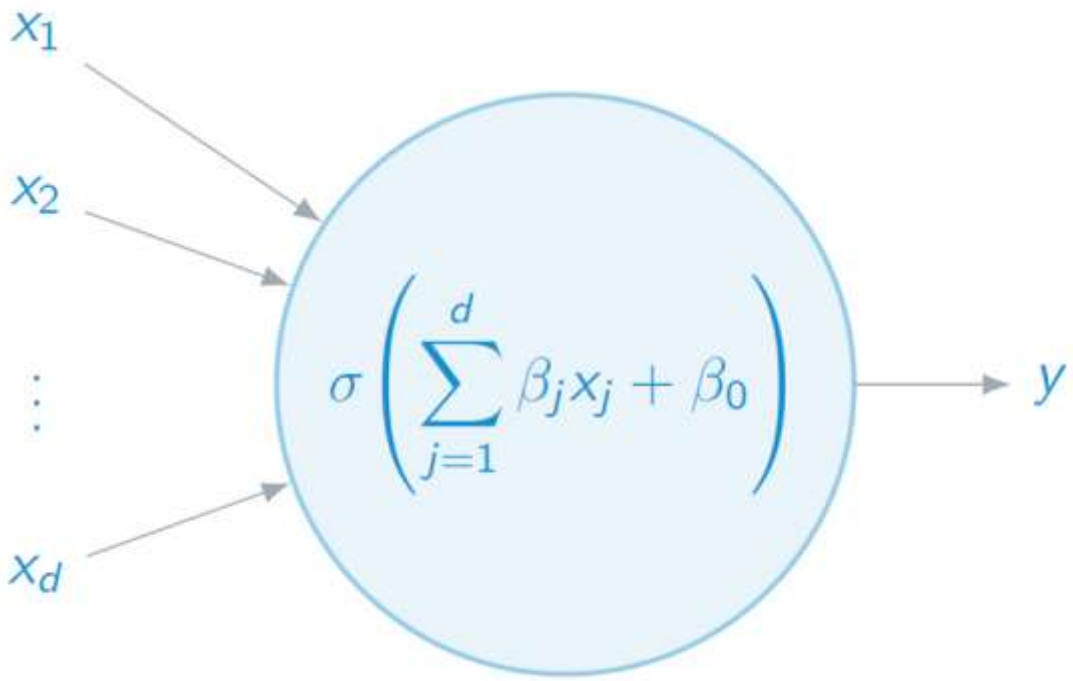
2012

- **Alex Krizhevsky, Ilya Sutskever, and Geoffrey Hinton**
  - Achieved 15.3% top-5 error in ImageNet 2012, versus 26.2% for the runner-up.
  - Key ingredients included convolutional networks, ReLU, dropout, data augmentation, and GPU training.
  - The result accelerated adoption of deep learning throughout computer vision.



Perceptron

Perceptron



Simple perceptron

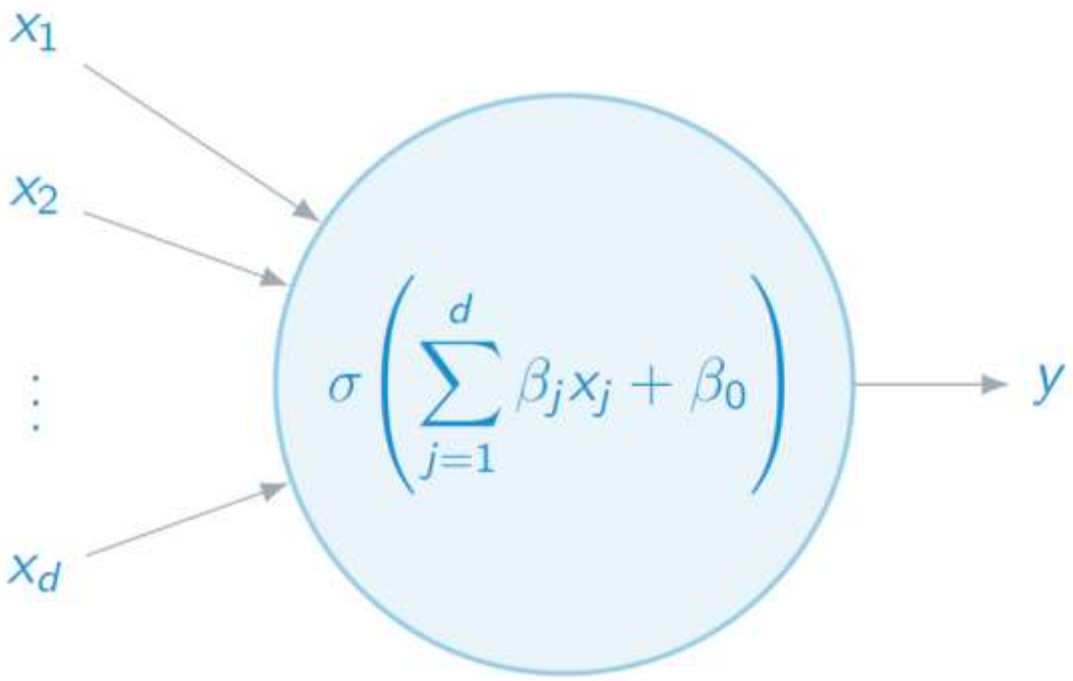
- The perceptron is the building block for neural networks.
- It was invented by Rosenblatt in 1957 at Cornell Labs, and first mentioned in the paper 'The Perceptron – a perceiving and recognizing automaton'.
- Perceptron computes a linear combination of factor of input and returns the sign.

Perceptron

x_j is the j-th feature of a sample and θ_j is the j-th weight. θ_0 is defined as the bias.

The bias alters the position of the decision boundary between the 2 classes.

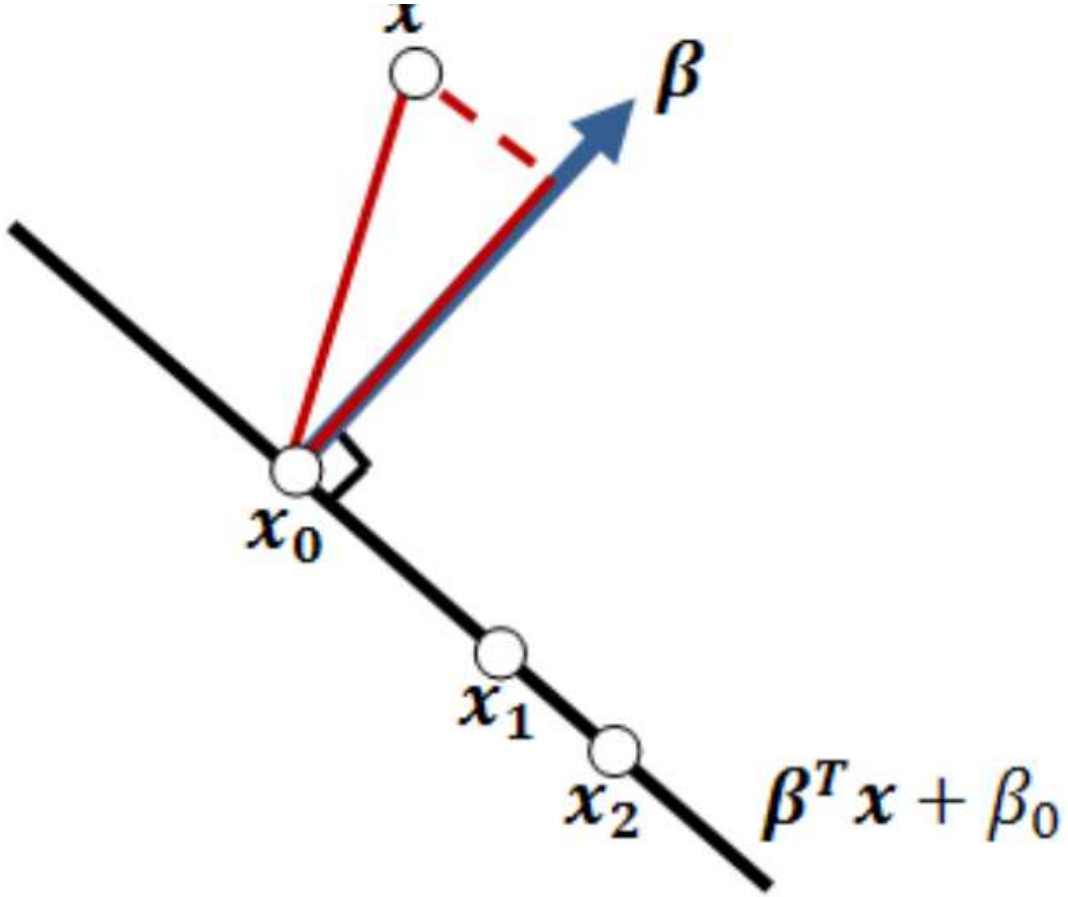
From a geometrical point of view, Perceptron assigns label "1" to elements on one side of $\theta^T x + \theta_0$ and label "−1" to elements on the other side.



Simple perceptron

Perceptron

- define a cost function, $\phi(\boldsymbol{\beta}, \beta_0)$, as a summation of the distance between all misclassified points and the hyper-plane, or the decision boundary.
- To minimize this cost function, we need to estimate $\boldsymbol{\beta}, \boldsymbol{\beta_0}$. $\min_{\beta,\beta_0} \phi(\boldsymbol{\beta}, \beta_0) = \{\text{distance of all misclassified points}\}$



Distance between the point and the decision boundary hyperplane (black line).

1) A hyper-plane L can be defined as

$$L = \{x : f(x) = \beta^T x + \beta_0 = 0\},$$

For any two arbitrary points x_1 and x_2 on L , we have

$$\beta^T x_1 + \beta_0 = 0 \ ,$$

$$\beta^T x_2 + \beta_0 = 0 \ ,$$

such that

$$\beta^T (x_1 - x_2) = 0 \ .$$

Therefore, β is orthogonal to the hyper-plane and it is the normal vector.

2) For any point x_0 in L,

$\beta^T x_0 + \beta_0 = 0$, which means $\beta^T x_0 = -\beta_0$.

3) We set $\beta^* = \frac{\beta}{||\beta||}$ as the unit normal vector of the hyper-plane L . For simplicity we call β^* norm vector. The distance of point x to L is given by

$$\beta^{*T}(x - x_0) = \beta^{*T}x - \beta^{*T}x_0 = \frac{\beta^T x}{||\beta||} + \frac{\beta_0}{||\beta||} = \frac{(\beta^T x + \beta_0)}{||\beta||}$$

Where x_0 is any point on L . Hence, $\beta^T x + \beta_0$ is proportional to the distance of the point x to the hyper-plane L .

4) The distance from a misclassified data point x_i to the hyper-plane L is

$$d_i = -y_i(\beta^T x_i + \beta_0)$$

where y_i is a target value, such that $y_i = 1$ if $\beta^T x_i + \beta_0 < 0$, $y_i = -1$ if $\beta^T x_i + \beta_0 > 0$

Since we need to find the distance from the hyperplane to the *misclassified* data points, we need to add a negative sign in front. When the data point is misclassified, $\beta^T x_i + \beta_0$ will produce an opposite sign of y_i . Since we need a positive sign for distance, we add a negative sign.

Learning Perceptron

The gradient descent is an optimization method that finds the minimum of an objective function by incrementally updating its parameters in the negative direction of the derivative of this function. That is, it finds the steepest slope in the D-dimensional space at a given point, and descends down in the direction of the negative slope. Note that unless the error function is convex, it is possible to get stuck in a local minima. In our case, the objective function to be minimized is classification error and the parameters of this function are the weights associated with the inputs, β

The gradient descent algorithm updates the weights as follows:

$$\beta^{\text{new}} \leftarrow \beta^{\text{old}} - \rho \frac{\partial Err}{\partial \beta}$$

ρ is called the *learning rate*.
The Learning Rate ρ is positively related to the step size of convergence of $\min \phi(\boldsymbol{\beta}, \beta_0)$.
i.e. the larger ρ is, the larger the step size is.
Typically, $\rho \in [0.1, 0.3]$.

The classification error is defined as the distance of misclassified observations to the decision boundary:

To minimize the cost function $\phi(\boldsymbol{\beta}, \beta_0) = -\sum_{i \in M} y_i(\boldsymbol{\beta}^T x_i + \beta_0)$ where

$M = \{\text{all points that are misclassified}\}$

$$\frac{\partial \phi}{\partial \boldsymbol{\beta}} = -\sum_{i \in M} y_i x_i \text{ and } \frac{\partial \phi}{\partial \beta_0} = -\sum_{i \in M} y_i$$

Therefore, the gradient is

$$\nabla D(\beta, \beta_0) = \begin{pmatrix} -\sum_{i \in M} y_i x_i \\ -\sum_{i \in M} y_i \end{pmatrix}$$

Using the gradient descent algorithm to solve these two equations, we have

$$\begin{pmatrix} \boldsymbol{\beta}^{\text{new}} \\ \beta_0^{\text{new}} \end{pmatrix} = \begin{pmatrix} \boldsymbol{\beta}^{\text{old}} \\ \beta_0^{\text{old}} \end{pmatrix} + \rho \begin{pmatrix} y_i x_i \\ y_i \end{pmatrix}$$

If the data is linearly-separable, the solution is theoretically guaranteed to converge to a separating hyperplane in a finite number of iterations.

In this situation the number of iterations depends on the learning rate and the margin. However, if the data is not linearly separable there is no guarantee that the algorithm converges.

Features

- A Perceptron can only discriminate between two classes at a time.
- When data is (linearly) separable, there are an infinite number of solutions depending on the starting point.
- Even though convergence to a solution is guaranteed if the solution exists, the finite number of steps until convergence can be very large.
- The smaller the gap between the two classes, the longer the time of convergence.

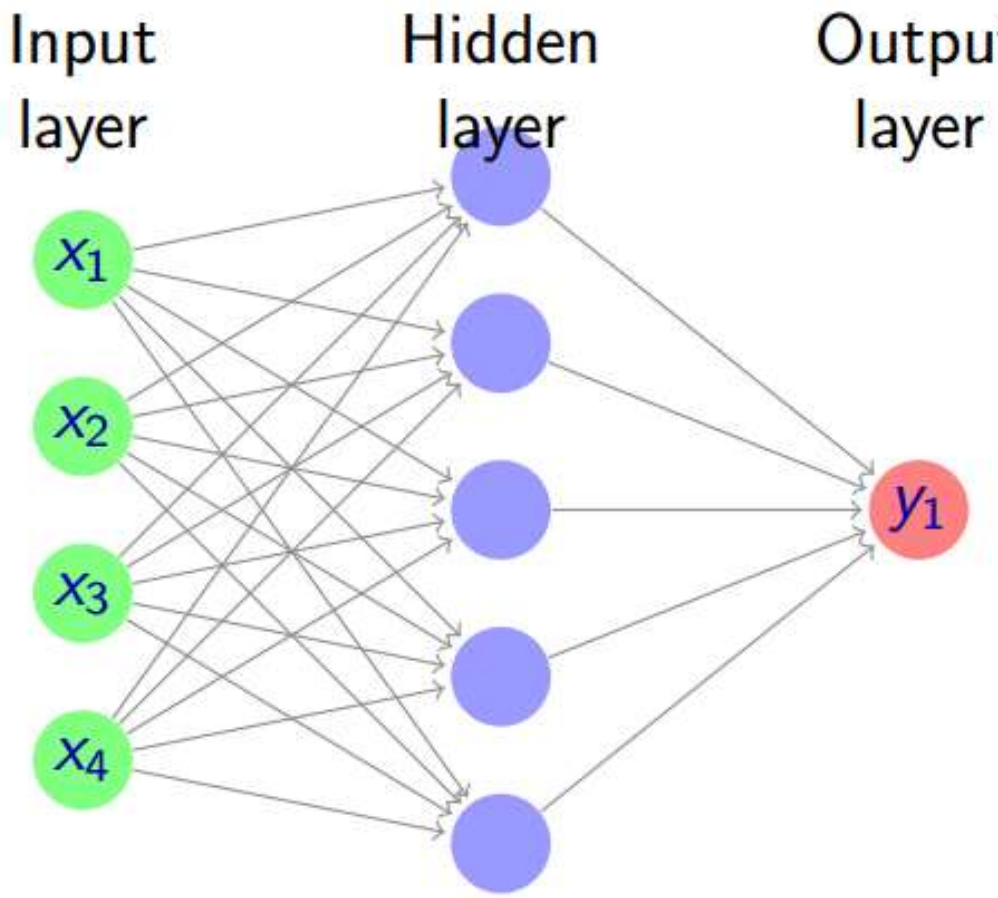
- When the data is not separable, the algorithm will not converge (it should be stopped after N steps).
- A learning rate that is too high will make the perceptron periodically oscillate around the solution unless additional steps are taken.
- The L.S compute a linear combination of feature of input and return the sign.
- This were called Perceptron in the engineering literate in late 1950.
- Learning rate affects the accuracy of the solution and the number of iterations directly.

Separability and convergence

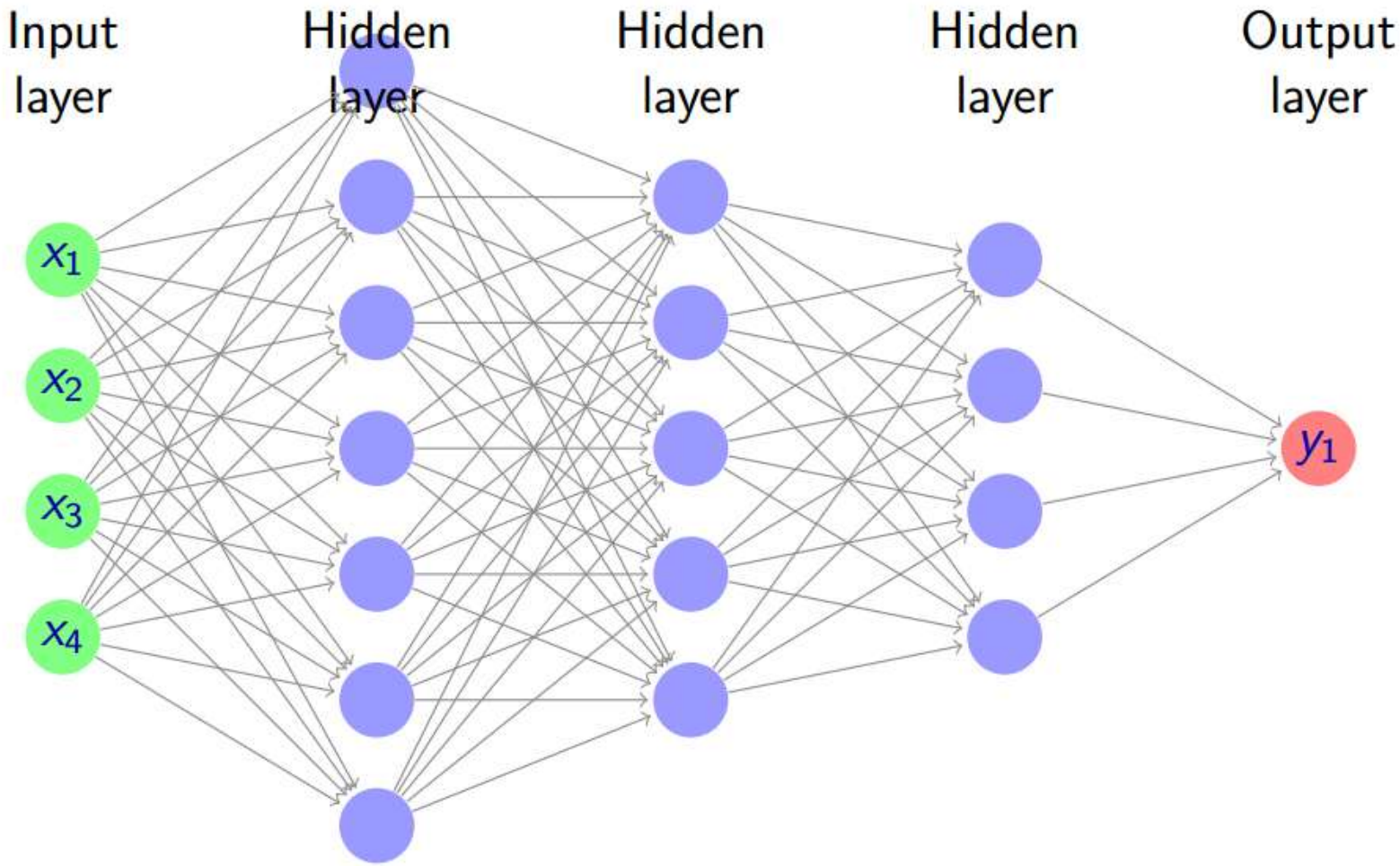
The training set D is said to be linearly separable if there exists a positive constant γ and a weight vector β such that $(\beta^T x_i + \beta_0) y_i > \gamma$ for all $1 < i < n$. That is, if we say that β is the weight vector of Perceptron and y_i is the true label of x_i, then the signed distance of the x_i from β is greater than a positive constant γ for any $(x_i, y_i) \in D$.

Feedforward neural networks

Feedforward Neural Network



Feedforward Deep Networks

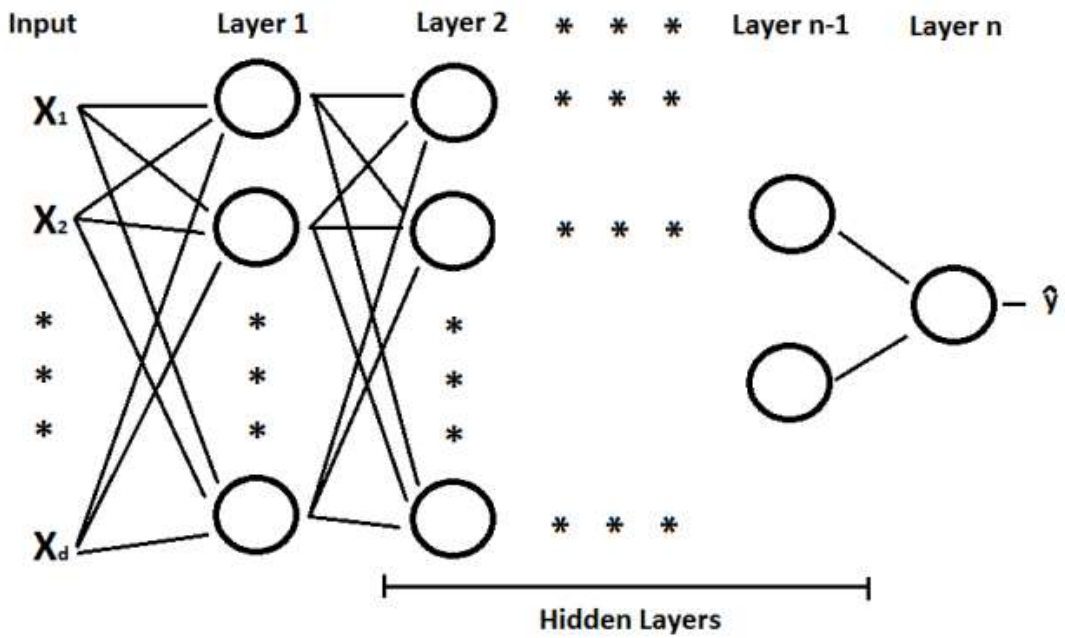


Feedforward Deep Networks

- Feedforward deep networks, a.k.a. multilayer perceptrons (MLPs), are parametric functions composed of several parametric functions.
- Each layer of the network defines one of these sub-functions.
- Each layer (sub-function) has multiple inputs and multiple outputs.
- Each layer composed of many units (scalar output of the layer).
- We sometimes refer to each unit as a feature.
- Each unit is usually a simple transformation of its input.
- The entire network can be very complex.

Feedforward Neural Network

- A neural network is a multistate regression model which is typically represented by a network diagram.



Feed Forward Neural Network

Feedforward Neural Network

- For regression, typically k = 1 (the number of nodes in the last layer), there is only one output unit y_1 at the end.
- For c-class classification, there are typically c units at the end with the c^{th} unit modelling the probability of class c, each y_c is coded as 0-1 variable for the cth class.

Feedforward Neural Network

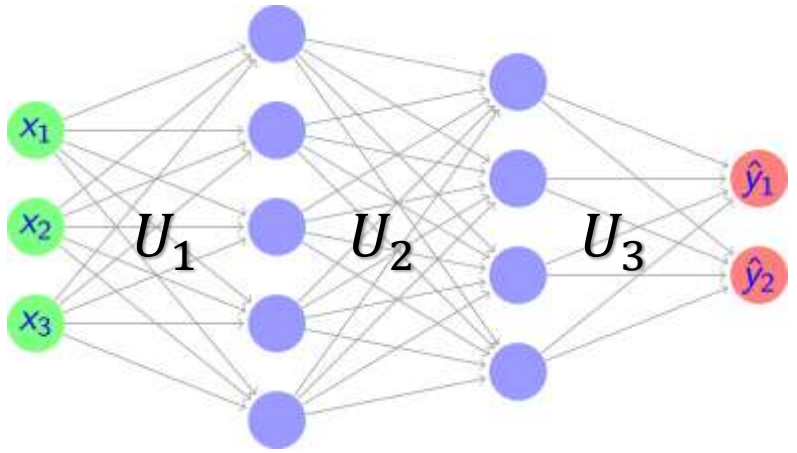
- Consider a feed forward neural network

$$U_1 = \begin{bmatrix} u_{11} & u_{12} & u_{13} & u_{14} & u_{15} \\ u_{21} & u_{22} & u_{23} & u_{24} & u_{25} \\ u_{31} & u_{32} & u_{33} & u_{34} & u_{35} \end{bmatrix}_{3\times 5}$$

$$U_2 = \begin{bmatrix} u_{11} & u_{12} & u_{13} & u_{14} \\ u_{21} & u_{22} & u_{23} & u_{24} \\ u_{31} & u_{32} & u_{33} & u_{34} \\ u_{41} & u_{42} & u_{43} & u_{44} \\ u_{51} & u_{52} & u_{53} & u_{54} \end{bmatrix}_{5\times 4}$$

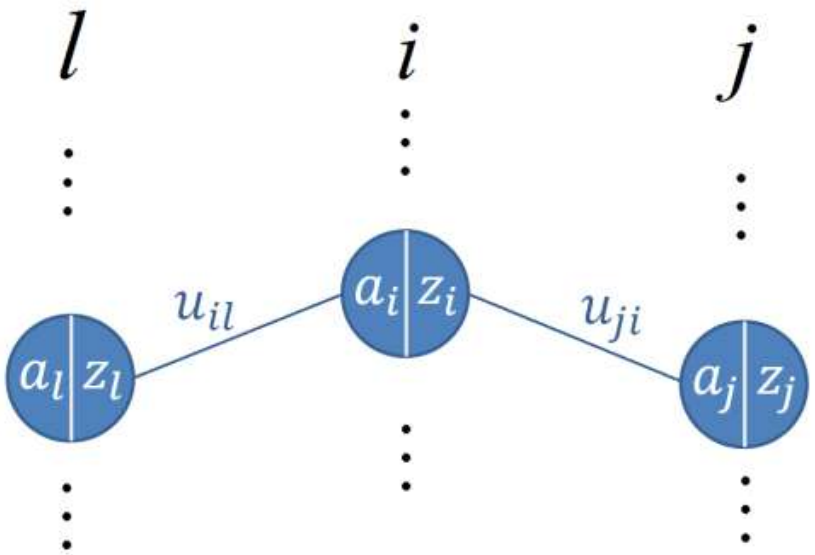
$$U_3 = \begin{bmatrix} u_{11} & u_{12} \\ u_{21} & u_{22} \\ u_{31} & u_{32} \end{bmatrix}_{4\times 2}$$

$$X = \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix}$$



Backpropagattion

Backpropagation

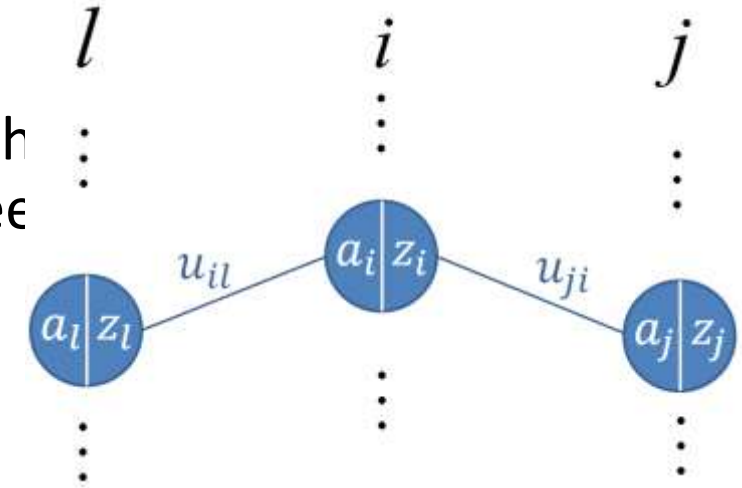


Nodes from three hidden layers within the neural network. Each node is divided into the weighted sum of the inputs and the output of the activation function.

$$a_i = \sum_l z_l u_{il}$$
$$z_i = \sigma(a_i)$$
$$\sigma(a) = \frac{1}{1+e^{-a}}$$

- Nodes from three hidden layers within the neural network are considered for the backpropagation algorithm.

- Each node has been divided into the weighted sum of th
output of the activation function z. The weights betwee
by u.

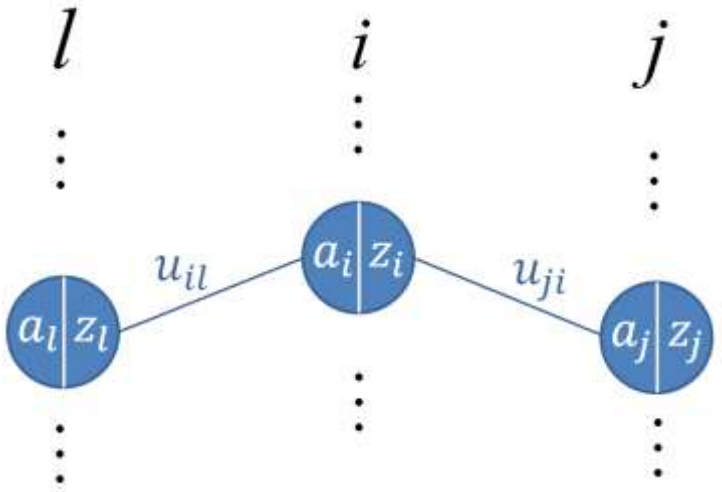


$$a_i = \sum_l z_l u_{il}$$

$$z_i = \sigma(a_i)$$

- Take the derivative w.r.t weight u_{il}

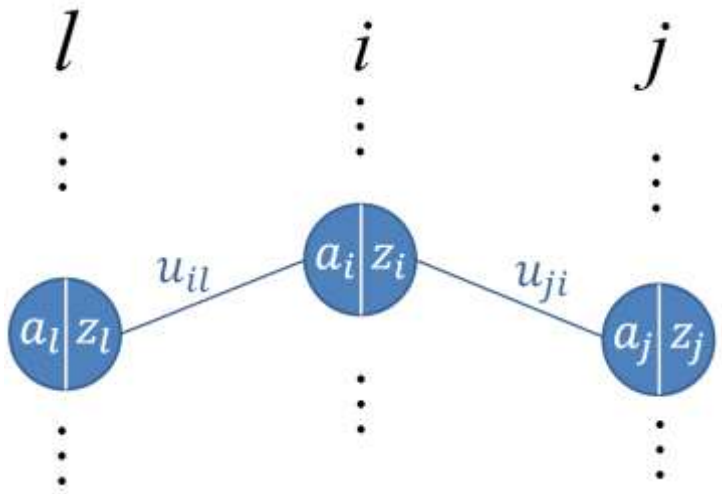
$$\frac{\partial |y - \hat{y}|^2}{\partial u_{il}} = \frac{\partial |y - \hat{y}|^2}{\partial a_i} \cdot \frac{\partial a_i}{\partial u_{il}}$$



- Take the derivative w.r.t weight u_{il}

Chain rule

$$\frac{\partial |y-\hat{y}|^2}{\partial u_{il}} = \frac{\partial |y-\hat{y}|^2}{\partial a_i} \cdot \frac{\partial a_i}{\partial u_{il}}$$

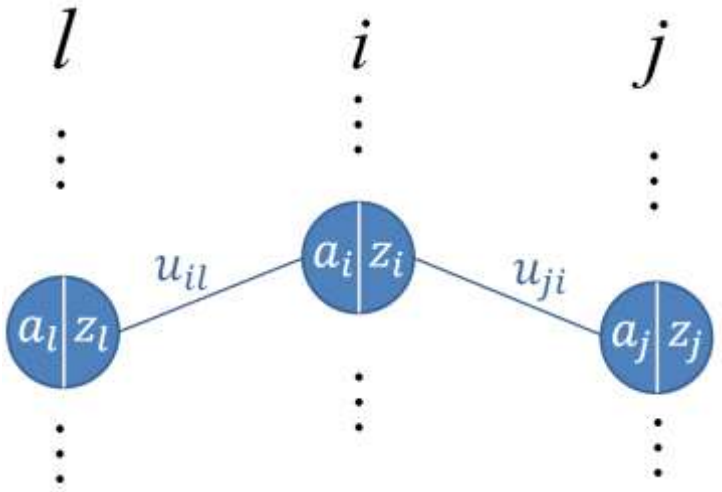


- Take the derivative w.r.t weight u_{il}

Chain rule

$$\frac{\partial |y-\hat{y}|^2}{\partial u_{il}} = \frac{\partial |y-\hat{y}|^2}{\partial a_i} \cdot \frac{\partial a_i}{\partial u_{il}}$$

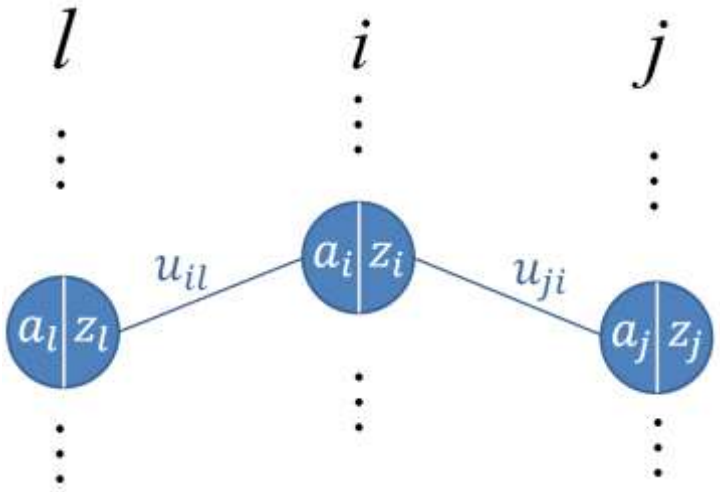
z_l



- Take the derivative w.r.t weight u_{il}

Chain rule

$$\frac{\partial |y-\hat{y}|^2}{\partial u_{il}} = \underbrace{\frac{\partial |y-\hat{y}|^2}{\partial a_i}}_{\delta_i} \cdot \underbrace{\frac{\partial a_i}{\partial u_{il}}}_{z_l}$$



- Take the derivative w.r.t weight u_{il}

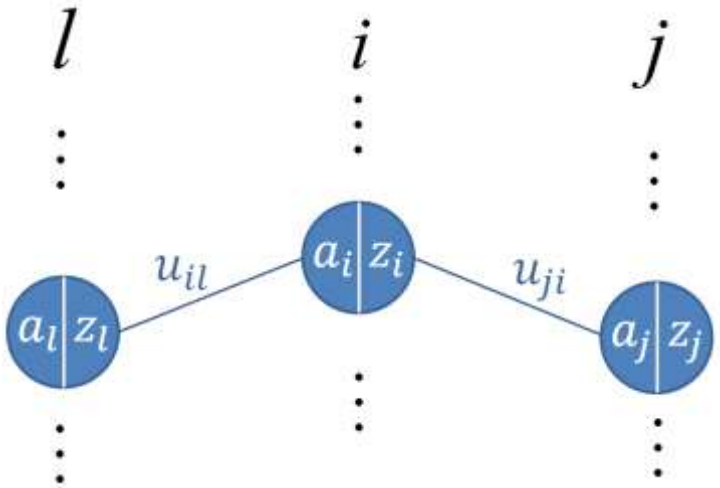
Chain rule

$$\frac{\partial |y-\hat{y}|^2}{\partial u_{il}} = \frac{\partial |y-\hat{y}|^2}{\partial a_i} \cdot \frac{\partial a_i}{\partial u_{il}}$$

$$\frac{\partial |y-\hat{y}|^2}{\partial a_i} = \delta_i \qquad \frac{\partial a_i}{\partial u_{il}} = z_l$$

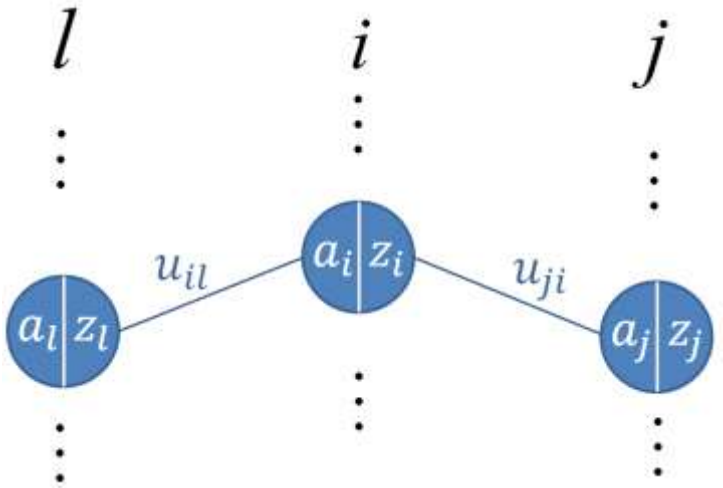
$$\frac{\partial |y-\hat{y}|^2}{\partial u_{il}} = \delta_i \cdot z_l$$

where $\delta_i = \frac{\partial |y-\hat{y}|^2}{\partial a_i}$



- whereas δ_i is unknown but can be expressed as a recursive definition in terms of δ_j.

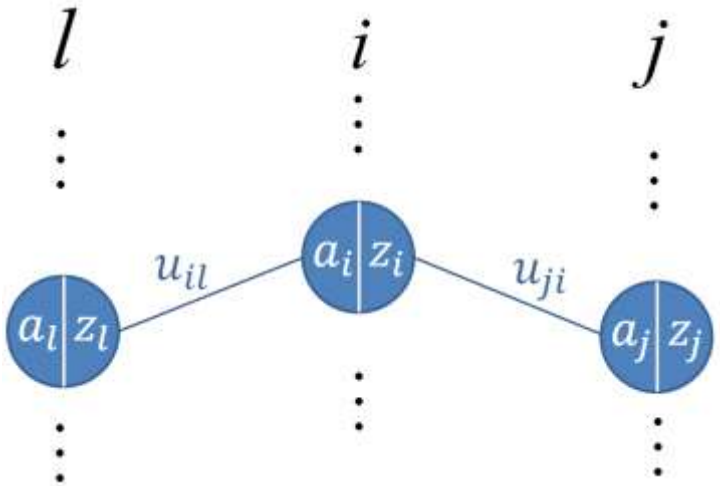
$$\delta_i = \frac{\partial |y - \hat{y}|^2}{\partial a_i} = \sum_j \frac{\partial |y - \hat{y}|^2}{\partial a_j} \cdot \frac{\partial a_j}{\partial a_i}$$



- whereas δ_i is unknown but can be expressed as a recursive definition in terms of δ_j.

Chain rule

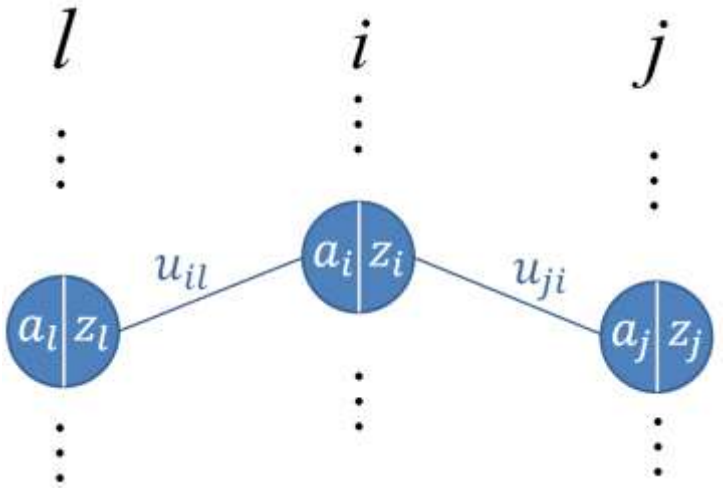
$$\delta_i = \frac{\partial |y - \hat{y}|^2}{\partial a_i} = \sum_j \frac{\partial |y - \hat{y}|^2}{\partial a_j} \cdot \frac{\partial a_j}{\partial a_i}$$



- whereas δ_i is unknown but can be expressed as a recursive definition in terms of δ_j.

Chain rule

$$\delta_i = \frac{\partial |y - \hat{y}|^2}{\partial a_i} = \sum_j \underbrace{\frac{\partial |y - \hat{y}|^2}{\partial a_j}}_{\delta_j} \cdot \frac{\partial a_j}{\partial a_i}$$

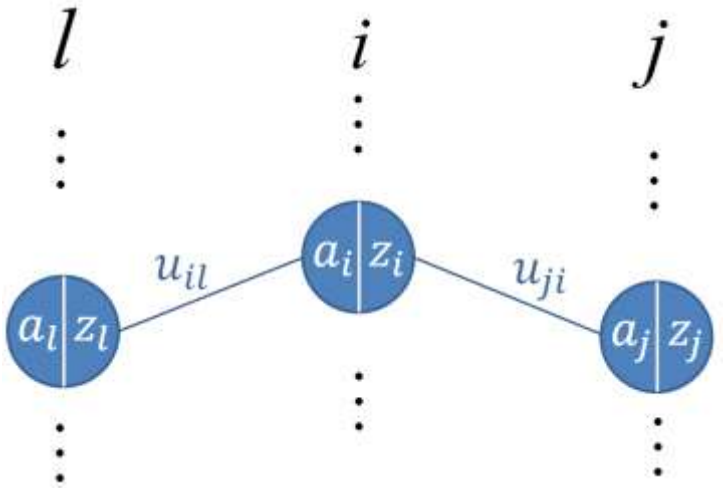


- whereas δ_i is unknown but can be expressed as a recursive definition in terms of δ_j.

Chain rule

$$\delta_i = \frac{\partial |y - \hat{y}|^2}{\partial a_i} = \sum_j \underbrace{\frac{\partial |y - \hat{y}|^2}{\partial a_j}}_{\delta_j} \cdot \underbrace{\frac{\partial a_j}{\partial a_i}}$$

$$\frac{\partial a_j}{\partial a_i} = \frac{\partial a_j}{\partial z_i} \cdot \frac{\partial z_i}{\partial a_i}$$

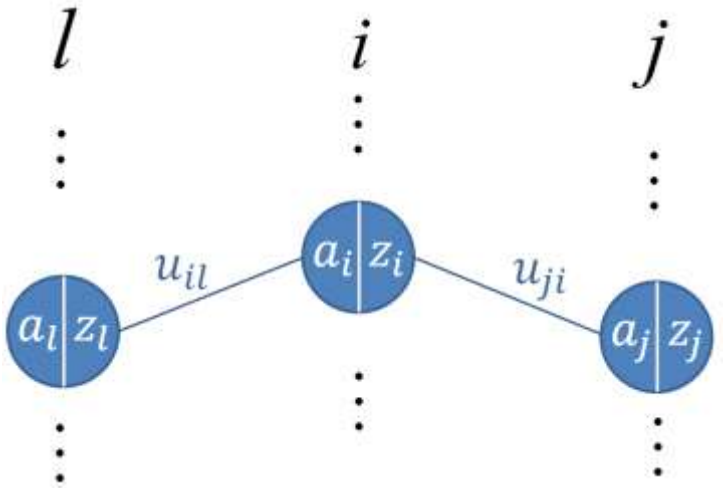


- whereas δ_i is unknown but can be expressed as a recursive definition in terms of δ_j.

Chain rule

$$\delta_i = \frac{\partial |y - \hat{y}|^2}{\partial a_i} = \sum_j \underbrace{\frac{\partial |y - \hat{y}|^2}{\partial a_j}}_{\delta_j} \cdot \underbrace{\frac{\partial a_j}{\partial a_i}}$$

$$\frac{\partial a_j}{\partial a_i} = \underbrace{\frac{\partial a_j}{\partial z_i}}_{u_{ji}} \cdot \frac{\partial z_i}{\partial a_i}$$

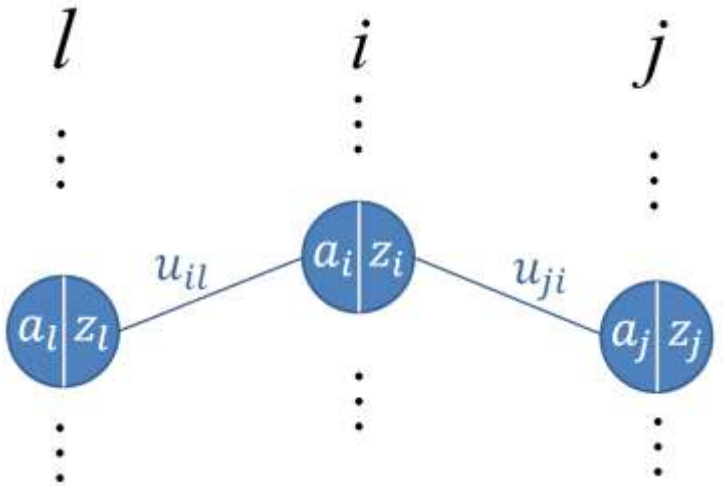


- whereas δ_i is unknown but can be expressed as a recursive definition in terms of δ_j.

Chain rule

$$\delta_i = \frac{\partial |y - \hat{y}|^2}{\partial a_i} = \sum_j \underbrace{\frac{\partial |y - \hat{y}|^2}{\partial a_j}}_{\delta_j} \cdot \underbrace{\frac{\partial a_j}{\partial a_i}}$$

$$\frac{\partial a_j}{\partial a_i} = \underbrace{\frac{\partial a_j}{\partial z_i}}_{u_{ji}} \cdot \underbrace{\frac{\partial z_i}{\partial a_i}}_{\sigma'(a_i)}$$

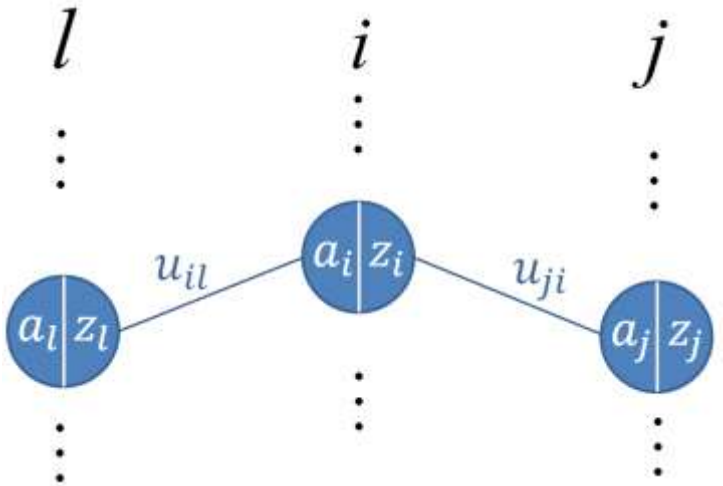


- whereas δ_i is unknown but can be expressed as a recursive definition in terms of δ_j.

Chain rule

$$\delta_i = \frac{\partial |y - \hat{y}|^2}{\partial a_i} = \sum_j \underbrace{\frac{\partial |y - \hat{y}|^2}{\partial a_j}}_{\delta_j} \cdot \underbrace{\frac{\partial a_j}{\partial a_i}}$$

$$\delta_i = \sum_j \delta_j \cdot \frac{\partial a_j}{\partial z_i} \cdot \frac{\partial z_i}{\partial a_i} \qquad \frac{\partial a_j}{\partial a_i} = \underbrace{\frac{\partial a_j}{\partial z_i}}_{u_{ji}} \cdot \underbrace{\frac{\partial z_i}{\partial a_i}}_{\sigma'(a_i)}$$



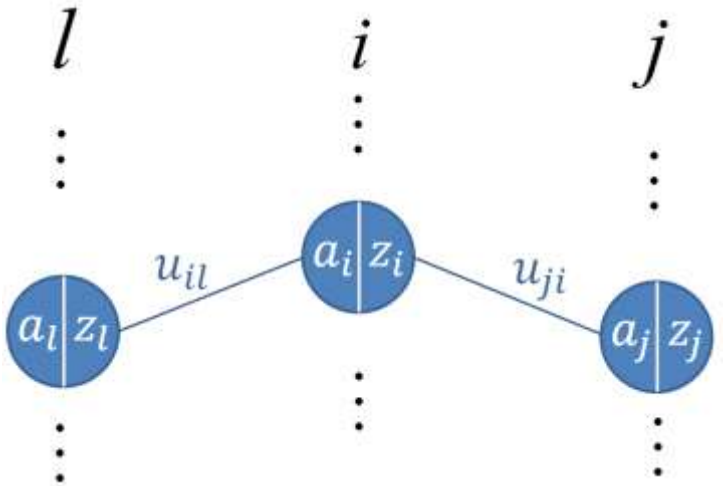
- whereas δ_i is unknown but can be expressed as a recursive definition in terms of δ_j.

Chain rule

$$\delta_i = \frac{\partial |y - \hat{y}|^2}{\partial a_i} = \sum_j \underbrace{\frac{\partial |y - \hat{y}|^2}{\partial a_j}}_{\delta_j} \cdot \underbrace{\frac{\partial a_j}{\partial a_i}}$$

$$\delta_i = \sum_j \delta_j \cdot \frac{\partial a_j}{\partial z_i} \cdot \frac{\partial z_i}{\partial a_i} \qquad \frac{\partial a_j}{\partial a_i} = \underbrace{\frac{\partial a_j}{\partial z_i}}_{u_{ji}} \cdot \underbrace{\frac{\partial z_i}{\partial a_i}}_{\sigma'(a_i)}$$

$$\delta_i = \sum_j \delta_j \cdot u_{ji} \cdot \sigma'(a_i)$$



- whereas δ_i is unknown but can be expressed as a recursive definition in terms of δ_j.

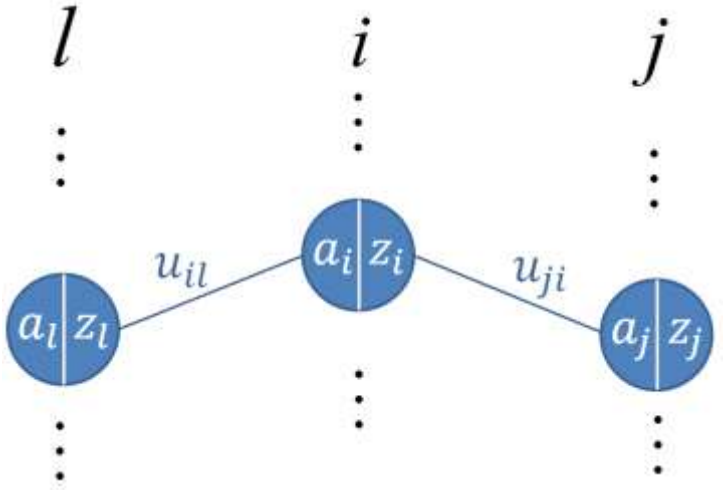
Chain rule

$$\delta_i = \frac{\partial |y - \hat{y}|^2}{\partial a_i} = \sum_j \underbrace{\frac{\partial |y - \hat{y}|^2}{\partial a_j}}_{\delta_j} \cdot \underbrace{\frac{\partial a_j}{\partial a_i}}$$

$$\delta_i = \sum_j \delta_j \cdot \frac{\partial a_j}{\partial z_i} \cdot \frac{\partial z_i}{\partial a_i} \qquad \frac{\partial a_j}{\partial a_i} = \underbrace{\frac{\partial a_j}{\partial z_i}}_{u_{ji}} \cdot \underbrace{\frac{\partial z_i}{\partial a_i}}_{\sigma'(a_i)}$$

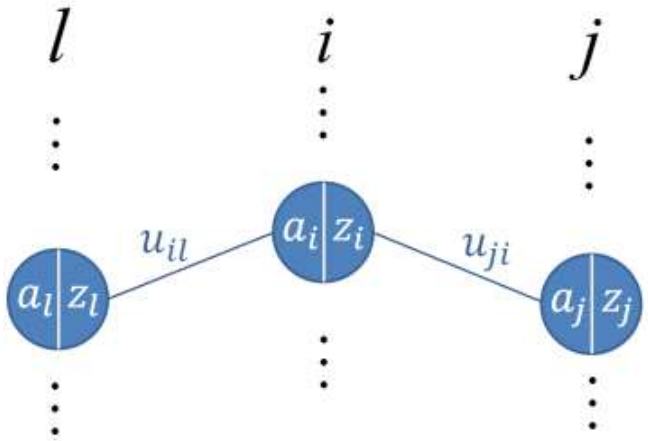
$$\delta_i = \sum_j \delta_j \cdot u_{ji} \cdot \sigma'(a_i)$$

$$\delta_i = \sigma'(a_i) \sum_j \delta_j \cdot u_{ji}$$



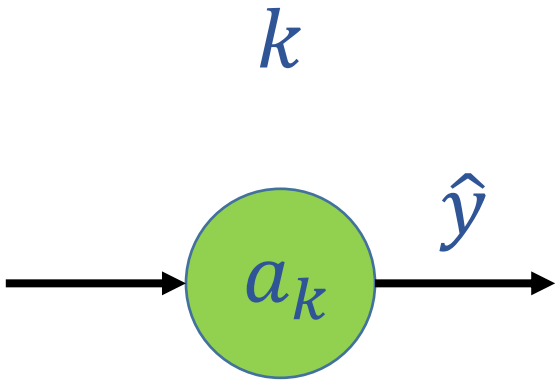
- The recursive definition of δ_i can be considered as a cost function at layer i for achieving the original goal of optimizing the weights to minimize $\|y - \hat{y}\|^2$

$$\delta_i = \sigma'(a_i) \sum_j \delta_j \cdot u_{ji}$$



Now consider

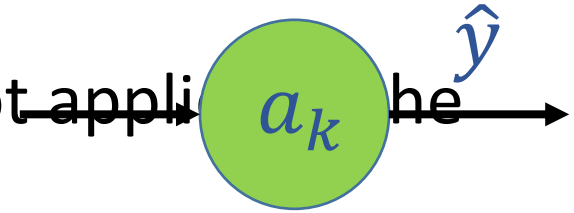
$$\delta_k = \frac{\partial \|y - \hat{y}\|^2}{\partial a_k}$$



$$\delta_k = \frac{\partial \|y - \hat{y}\|^2}{\partial a_k}$$

k

- Where $a_k = \hat{y}$ because an activation function is not applied to the output layer



$$\delta_k = \frac{\partial \|y - \hat{y}\|^2}{\partial \hat{y}} = -2\|y - \hat{y}\|$$

- Since y is known and $\hat{y}$ can be computed for each d

$$\delta_i^{(k)} = -2|y_i - \hat{y}_i|$$

- assuming small, random, initial values for the
  network in the beginning.

$$\delta_i^{(k-1)} = \sigma'(a_i^{(k-1)}) \sum_j \delta_j^{(k)} \cdot u_{ij}^{(k-1)}$$

- Therefore the δ values for the layer before t
  computed using δ_k and then the δ values fo
  layer before the output layer can be computed and so on.

$$\delta_i^{(k-2)} = \sigma'(a_i^{(k-2)}) \sum_j \delta_j^{(k-1)} \cdot u_{ij}^{(k-2)}$$

- Once all δ values are known, the errors due to each of the weights u will be known and techniques like gradient descent can be used to optimize the weights.

$$u_{il}^{new} \leftarrow u_{il}^{old} - \rho \frac{\partial \|y - \hat{y}\|^2}{\partial u_{il}}$$

Backpropagation

Backpropagation procedure is done using the following steps:

- First arbitrarily choose some random weights (preferably close to zero) for your network.
- Apply x to the FFNN's input layer, and calculate the outputs of all input neurons.
- Propagate the outputs of each hidden layer forward, one hidden layer at a time, and calculate the outputs of all hidden neurons.
- Once x reaches the output layer, calculate the output(s) of all output neuron(s) given the outputs of the previous hidden layer.
- At the output layer, compute $\delta_k = -2(y_k - \hat{y}_k)$ for each output neuron(s).

- Compute each δ_i , starting from $i = k - 1$ all the way to the first hidden layer, where $\delta_i = \sigma'(a_i) \sum_j \delta_j \cdot u_{ji}$.

- Compute $\frac{\partial \|y - \hat{y}\|^2}{\partial u_{il}} = \delta_i z_l$ for all weights u_{il} .

- Then update $u_{il}^{\text{new}} \leftarrow u_{il}^{\text{old}} - \rho \cdot \frac{\partial \|y - \hat{y}\|^2}{\partial u_{il}}$ for all weights u_{il} .

- Continue for next data points and iterate on the training set until weights converge.

Epochs

It is common to cycle through the all of the data points multiple times in order to reach convergence. An epoch represents one cycle in which you feed all of your datapoints through the neural network. It is good practice to randomized the order you feed the points to the neural network within each epoch; this can prevent your weights changing in cycles. The number of epochs required for convergence depends greatly on the learning rate & convergence requirements used.